

目录

1	模块一	1
1.1	1. 约瑟夫问题	1
1.1.1	任务	1
1.1.2	1. 需求分析	1
1.1.3	2. 概要设计	1
1.1.4	3. 算法设计（流程图）	2
1.1.5	4. 详细设计	2
1.1.6	顺序存储结构实现	3

1 模块一

1.1 1. 约瑟夫问题

1.1.1 任务

1.1.2 1. 需求分析

分析总结逻辑结构 本质上是线性结构上的循环删除操作。

1. 初始有 n 只猴子，按顺序围成一圈；
2. 从第 1 只开始按顺时针顺序报数；
3. 每报到第 m 只猴子，该猴子执行出圈（删除操作）；
4. 剩余猴子继续顺时针循环报数；
5. 持续循环，直至圈内仅剩余 1 只猴子，该猴子即为最终大王。

逻辑结果

- 数据之间具备线性邻接关系；
- 逻辑上构成环状结构；
- 核心操作：循环遍历、删除当前节点。

分析总结运算集合 遍历、计数、标记删除、循环跳转、判停（剩余元素数量为 1）。

1.1.3 2. 概要设计

存储结构设计 本问题采用两种存储结构分别实现：

1. **顺序结构设计** 采用一维顺序表模拟环形结构：

- `int a[N]`：存储猴子编号（取值 $1 \sim n$ ）；
- `int st[N]`：状态标记数组；
- `st[i] = 0`：表示猴子未出圈；

- $st[i] = 1$: 表示猴子已出圈;
- 通过取模运算 $(index + 1) \% n$ 实现环形循环访问。

特点:

- 结构简单, 逻辑直观、易于理解;
- 依赖辅助标记顺序表模拟删除;
- 时间复杂度较高, 需要反复遍历跳过已出圈元素。

2. 链式结构设计 采用循环单链表表示约瑟夫环:

结构特点:

- 每个链表结点存储一只猴子的编号;
- 链表尾结点指向头结点, 形成闭环循环链表;
- 通过指针循环移动完成报数计数;
- 修改指针指向完成结点删除操作。

优势:

- 存储模型贴合约瑟夫环环形问题本质;
- 找到目标结点后删除操作时间复杂度 $O(1)$;
- 无需额外辅助标记顺序表。

1.1.4 3. 算法设计 (流程图)

本问题分别给出顺序表模拟、循环单链表两种实现思路流程图, 下面插入流程图图片。

1.1.5 4. 详细设计

源代码 (带注释) 采用 左右双栏并排展示两种实现代码, 左侧为顺序表模拟, 右侧为循环单链表实现, 阅读顺序为先左后右。

1.1.6 顺序存储结构实现

代码:

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>

#define N 10000

int a[N], n, m;
int st[N];

int josephus_circle()
{
    // 如果只有一个人
    if (n == 1)
        return a[0];

    int num = n;
    int cnt = 0;
    int index = 0;

    while (num != 1)
    {
        // 已出圈则跳过
        if (st[index] == 1)
        {
            index = (index + 1) % n;
            continue;
        }

        cnt++;

        // 数到第m个
```

```
        if (cnt == m)
        {
            st[index] = 1;
            cnt = 0;
            num--;
        }

        index = (index + 1) % n;
    }

    // 找最后剩余的人
    for (int i = 0; i < n; i++)
    {
        if (st[i] == 0)
            return a[i];
    }

    return -1;
}

int main()
{
    printf("请输入n和m: ");

    scanf("%d%d", &n, &m);

    memset(st, 0, sizeof(st));

    for (int i = 0; i < n; i++)
        a[i] = i + 1;

    int ret = josephus_circle();
```

```
    printf("最后的大王编号为: %d\n", ret);

    return 0;
}
```